PES UNIVERSITY
Department of Computer Science and Engineering
DATABASE MANAGEMENT SYSTEM

INTELLIGENT DATABASE DISCOVERY AND OPTIMIZATION PLATFORM

Experiential Learning – Level 3
DBMS Mini Project Report

SRN	Student Name
PES1UG24AM191	Piyush Shiv
PES1UG24AM215	Raghav G Uddihal

Academic Project Submission | November 2026

1. Project Overview

Project title: Intelligent Database Discovery and Optimization Platform (IDOP)

IDOP is a web-based platform for exploring relational database schemas, collecting and analyzing SQL workloads, discovering related queries through semantic search, and reviewing optimization recommendations with execution-plan information. It combines MySQL for structured storage and query operations with Qdrant for vector similarity search.

The application provides a Next.js interface backed by a FastAPI service. Python modules implement SQL workload analysis and semantic-search orchestration. FastEmbed generates vector embeddings from workload descriptions and SQL text; Qdrant retrieves semantically similar workloads, while MySQL stores the associated workload and recommendation records.

Problem addressed. Database teams need a convenient way to inspect schemas and workloads, identify potentially inefficient query patterns, and find related SQL statements even when a user does not know the exact query text. IDOP brings these activities into one role-aware application.

Objectives.

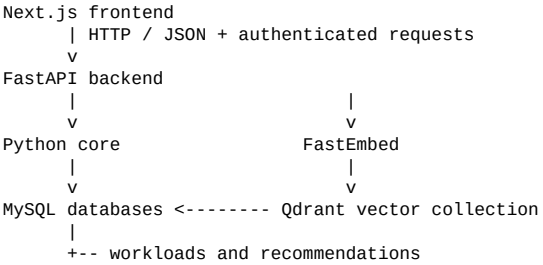
• Maintain structured database, schema, workload, recommendation, and audit metadata in MySQL.
• Analyze SQL workload patterns and produce explainable recommendations.
• Support natural-language workload discovery using vector embeddings and similarity search.
• Present workload details and MySQL execution plans through a web interface.
• Enforce role-based access for Viewer, Analyst, and Admin users.

2. Technology Stack

Technology	Role in the project
MySQL 8.4.11	Relational database engine for demo and platform databases, constraints, SQL operations, indexes, routines, and EXPLAIN.
Python	Workload analysis, database access, workload processing, and semantic-search orchestration.
FastAPI / Uvicorn	Backend REST API and ASGI server.
Qdrant	Vector database storing embeddings and payloads for semantic workload retrieval.
FastEmbed	Embedding generation using sentence-transformers/all-MiniLM-L6-v2.
Next.js / React / TypeScript	Frontend application and page routing.
Docker	Local Qdrant service runtime.
mysql-connector-python / python-dotenv	MySQL connectivity and environment configuration.

3. System Architecture

The frontend communicates with the FastAPI backend over HTTP. The backend uses the Python core modules to access MySQL and to coordinate semantic retrieval through FastEmbed and Qdrant.



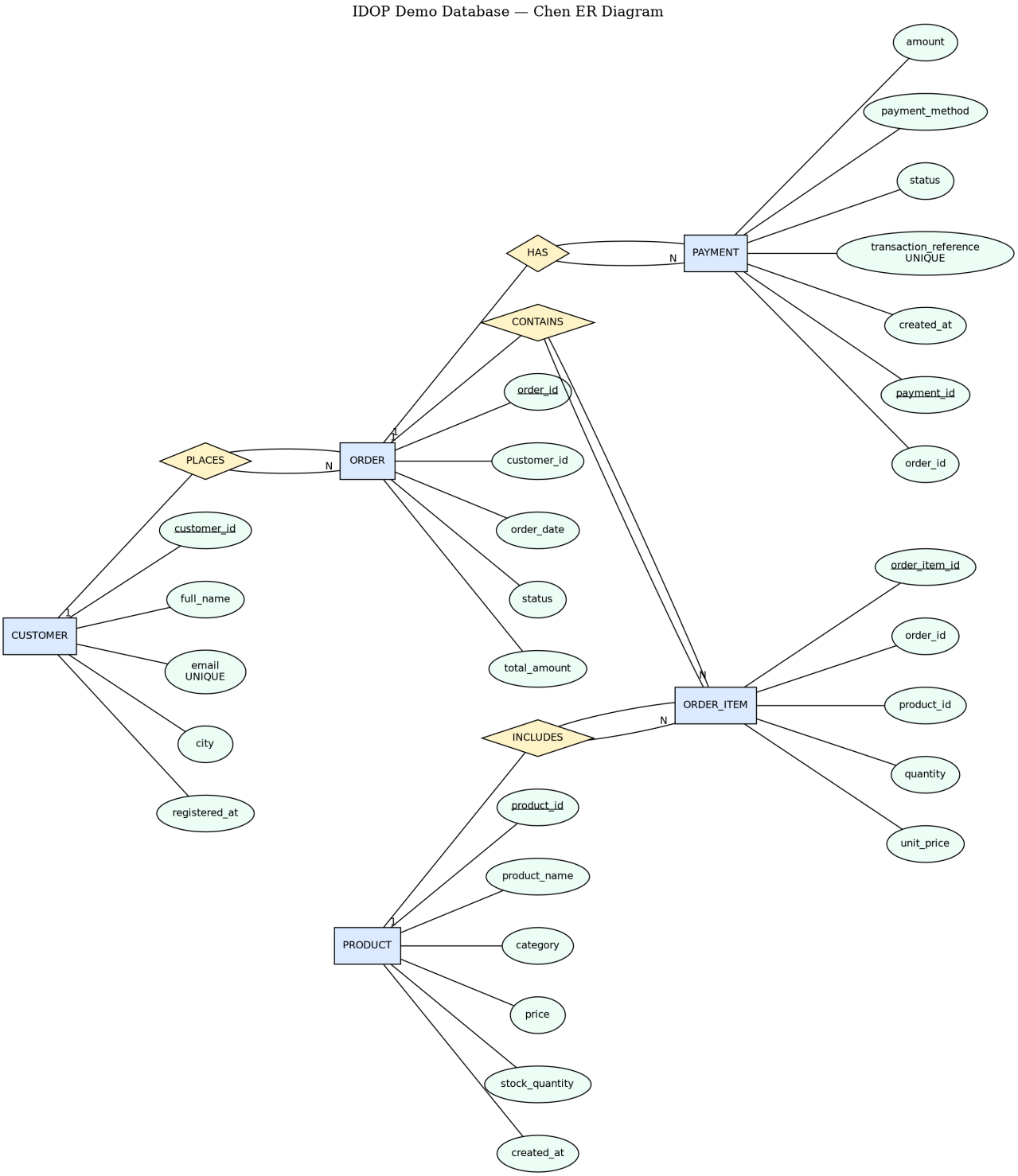
Relational databases. *idop_demo_db* contains the e-commerce demonstration tables: customers, products, orders, order_items, and payments. *idop_platform_db* stores users, data_sources, schema_objects, query_workloads, optimization_recommendations, and audit_logs.

Semantic retrieval flow. A natural-language search is embedded using FastEmbed. Qdrant returns the nearest workload vectors and their payloads. The workload identifiers connect the retrieved results to structured MySQL workload and recommendation records.

4. Relational Database Design

4.1 Demo Database — idop_demo_db

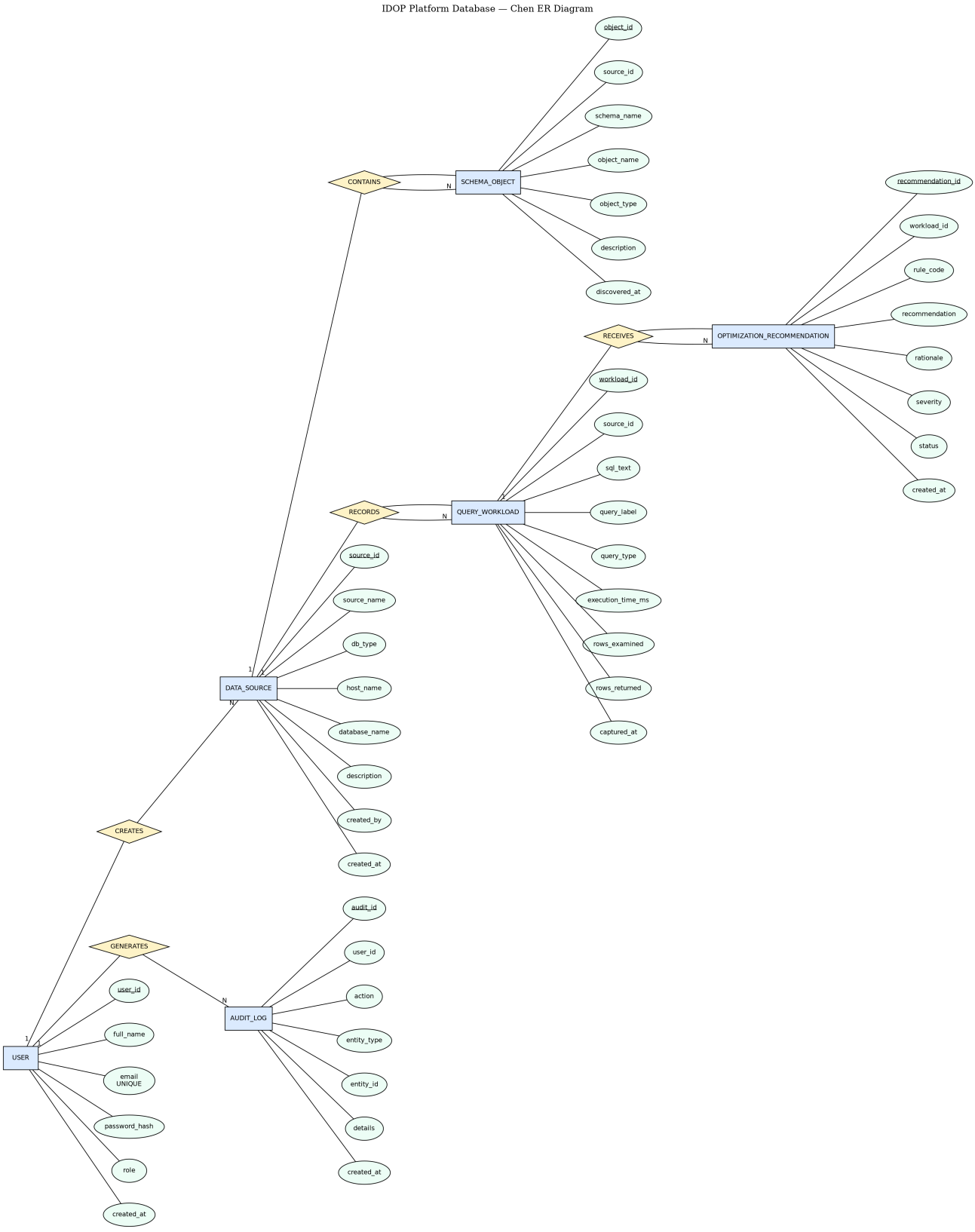
The conceptual ER model captures the e-commerce entities and their relationships. Primary keys are underlined; relationship cardinalities are shown at both ends.



Relationships: CUSTOMER 1:N ORDER; ORDER 1:N ORDER_ITEM; PRODUCT 1:N ORDER_ITEM; ORDER 1:N PAYMENT.

4.2 Platform Database — idop_platform_db

The platform ER model represents application users, registered data sources, discovered schema objects, captured workloads, optimization recommendations, and audit events.



Relationships: USER 1:N DATA_SOURCE; DATA_SOURCE 1:N SCHEMA_OBJECT; DATA_SOURCE 1:N QUERY_WORKLOAD; QUERY_WORKLOAD 1:N OPTIMIZATION_RECOMMENDATION; USER 1:N AUDIT_LOG. Nullable foreign keys represent optional

Demo database – idop_demo_db

Table	Description
customers	Customer information; one customer can place multiple orders.
products	Product catalogue referenced by order_items.
orders	Customer orders; references customers.
order_items	Associative order/product records with quantity and unit price.
payments	Payment records associated with orders.

Platform database – idop_platform_db

Table	Description
users	Application users, password hashes, and roles.
data_sources	Registered database sources.
schema_objects	Discovered database objects and schema metadata.
query_workloads	Captured SQL text, labels, and workload information.
optimization_recommendations	Recommendations associated with analyzed workloads.
audit_logs	Audit events, including recommendation status changes.

The schemas use primary keys, foreign keys, unique constraints, check constraints, and indexes to maintain data integrity and support lookups.

5. SQL – Database Commands and Operations

The implementation demonstrates DDL and DML, relational joins, aggregation, nested queries, views, stored functions, stored procedures, triggers, transactions, FULLTEXT search, and execution-plan inspection.

5.1 DDL and DML

```
CREATE TABLE customers (
  customer_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(120) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  city VARCHAR(100) NOT NULL,
  registered_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

SELECT * FROM customers;
UPDATE orders
SET status = 'CANCELLED'
WHERE order_id = 8;
```

5.2 Join and aggregation

```
SELECT c.full_name,
       COUNT(o.order_id) AS total_orders,
       SUM(o.total_amount) AS total_order_value
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.full_name;
```

The demonstrated query returned 10 customer rows and used LEFT JOIN, COUNT, SUM, and GROUP BY.

5.3 Nested query example

```
SELECT order_id, customer_id, total_amount, status
FROM orders
WHERE total_amount >
      (SELECT AVG(total_amount) FROM orders)
ORDER BY total_amount DESC;
```

5.4 View, function, and stored procedure

```
SELECT *
FROM idop_demo_db.v_customer_order_summary
LIMIT 10;

SELECT idop_demo_db.fn_order_item_total(1);

CALL idop_demo_db.sp_reconcile_order_total(1);
```

The demonstrated function returned 4798.00 for order 1. The stored procedure completed successfully. The customer-order summary view returned the customer-level order count, non-cancelled order value, and last order date.

5.5 Trigger and audit logging

An AFTER UPDATE trigger on optimization_recommendations inserts an audit record when a recommendation status changes. The audit log demonstration shows a RECOMMENDATION_STATUS_CHANGED event, the recommendation identifier, and JSON details containing the old and new statuses. The separate SHOW TRIGGERS terminal capture was omitted because its output was poorly formatted; the trigger's audit result is shown instead.

5.6 Transactions

```
START TRANSACTION;

UPDATE orders
SET status = 'CANCELLED'
WHERE order_id = 8;

SELECT order_id, status
FROM orders
WHERE order_id = 8;

ROLLBACK;

SELECT order_id, status
FROM orders
WHERE order_id = 8;
```

The demonstrated transaction changed order 8 to CANCELLED inside the transaction and restored it to PENDING after ROLLBACK.

5.7 FULLTEXT search and EXPLAIN

```
SELECT workload_id, query_label,
       MATCH(query_label, sql_text)
       AGAINST ('customer orders' IN NATURAL LANGUAGE MODE)
       AS relevance
FROM query_workloads
WHERE MATCH(query_label, sql_text)
      AGAINST ('customer orders' IN NATURAL LANGUAGE MODE)
ORDER BY relevance DESC;

EXPLAIN
SELECT *
FROM orders
WHERE customer_id = 1;
```

The FULLTEXT query returned three matching workload rows. EXPLAIN showed use of idx_orders_customer_date for the customer_id predicate. EXPLAIN reports the optimizer's execution plan and estimated rows; it is not itself a measured benchmark.

6. NoSQL / Vector Database – Qdrant

Qdrant is used as the vector database for semantic discovery of SQL workloads. The collection is named *sql_workloads*, uses cosine distance, and stores workload metadata as point payloads. The captured collection response reports four points and a vector size of 384.

6.1 Start Qdrant

```
docker run -d --name idop-qdrant \
  -p 6333:6333 -p 6334:6334 \
  -v idop_qdrant_data:/qdrant/storage qdrant/qdrant
```

6.2 Collection configuration and upsert – representative Python operations

```
COLLECTION = "sql_workloads"

vectors_config = models.VectorParams(
    size=vector_size,
    distance=models.Distance.COSINE
)

client.upsert(
    collection_name=COLLECTION,
    points=[
        models.PointStruct(
            id=workload_id,
            vector=embedding,
            payload={
                "workload_id": workload_id,
```

```
                "query_label": query_label,
                "sql_text": sql_text
            }
        )
    ],
    wait=True
)
```

6.3 Similarity search – representative Python operation

```
query_vector = next(
    embedding_model.query_embed(user_query)
).tolist()

results = client.query_points(
    collection_name="sql_workloads",
    query=query_vector,
    limit=limit,
    with_payload=True
).points
```

The captured Qdrant API response confirms collection status green, cosine distance, vector size 384, and four stored points. The application screenshot demonstrates natural-language search results with similarity scores.

7. Next.js – Project Tree Structure

```
frontend/
|-- app/
|   |-- admin/users/page.tsx
|   |-- api/admin/users/route.ts
|   |-- dashboard/page.tsx
|   |-- queries/[id]/page.tsx
|   |-- schemas/page.tsx
|   |-- search/page.tsx
|   |-- signup/page.tsx
|   |-- workloads/page.tsx
|   |-- layout.tsx
|   `-- page.tsx
|-- components/Navbar.tsx
|-- lib/api.ts
|-- next.config.ts
|-- package.json
`-- tsconfig.json
```

Page	Function
Login / Signup	Authentication and Viewer account registration.
Dashboard	Summary of workloads, databases, recommendations, and system status.
Schemas	Database and table metadata exploration.
Workloads	Captured SQL workload listing and analysis entry point.
Semantic Search	Natural-language search for related SQL workloads.
Query Details	Workload details, recommendations, and execution-plan information.
User Management	Administrator interface for account provisioning.

8. Authentication and Authorization

The application supports three roles: Viewer, Analyst, and Admin. Passwords are stored as hashes, and authenticated requests use signed tokens. Authorization checks are enforced by the backend rather than relying only on frontend navigation.

Role	Access
Viewer	Dashboard, schema exploration, and workload viewing.
Analyst	Viewer access plus semantic search and recommendation analysis/status operations.
Admin	Analyst access plus user-management and administrative provisioning.

The supplied UI evidence shows Viewer, Analyst, and Admin role indicators and the Admin user-management interface.

9. Testing and Demonstration Evidence

Area	Evidence captured / recorded result
Frontend	Login, dashboard, schema explorer, workload explorer, semantic search, and user-management pages captured.

Area	Evidence captured / recorded result
MySQL schemas	SHOW TABLES confirms six listed objects in each database, including the demo view.
Join and aggregation	Customer order summary query returned 10 rows.
Stored function and procedure	Function returned 4798.00; procedure completed with Query OK.
View	Customer summary view returned 10 rows.
Trigger / audit	Audit log contains a recommendation status-change event.
Transaction	ROLLBACK restored order 8 to PENDING.
FULLTEXT	Search returned three matching workloads.
EXPLAIN	Optimizer selected idx_orders_customer_date; estimated three rows.
Qdrant	Collection healthy, vector size 384, four points.

The project implementation summary also records 10 passing Python tests and a successful Next.js production build. These are implementation test results; the screenshots in this report are direct demonstration evidence.

10. Evaluation Rubric – Team Self-Assessment

The following status table maps the implemented project features to the DBMS evaluation areas. Statuses describe the team's implementation and evidence, not marks awarded by the evaluator.

Evaluation area	Status	Evidence / note
ER diagram and relational design	Implemented	Relational entities and relationships are represented in the project schema/ERD sources; include the rendered ERD in the final repository.
Relational schema, keys, relationships, constraints	Implemented	Two MySQL databases; primary/foreign keys and integrity constraints.
SQL DDL and DML	Implemented	Schema creation, seed data, SELECT, UPDATE, and other DML operations.
Joins and aggregate queries	Implemented	LEFT JOIN, COUNT, SUM, GROUP BY demonstrated.
Nested queries	Implemented	Scalar subquery using AVG demonstrated.
Views	Implemented	v_customer_order_summary queried successfully.
Stored procedures and functions	Implemented	sp_reconcile_order_total and fn_order_item_total executed.
Triggers and audit	Implemented	AFTER UPDATE trigger and resulting audit event demonstrated.
Transactions	Implemented	START TRANSACTION, UPDATE, ROLLBACK demonstrated.
Indexes and query-plan analysis	Implemented	EXPLAIN uses idx_orders_customer_date; FULLTEXT index/search demonstrated.
NoSQL / vector database	Implemented	Qdrant collection, four stored points, cosine similarity, and semantic-search UI.
Next.js interface	Implemented	Dashboard, schema, workload, search, and admin pages captured.
Backend and database integration	Implemented	FastAPI backend integrates Python core, MySQL, and Qdrant.
Authentication and role-based access	Implemented	Viewer, Analyst, and Admin roles supported.
Documentation and GitHub	Implemented	Project README and source repository: github.com/PiyushShiv26/DBMS-mini-project.
CTE / correlated / recursive query evidence	Verify against submitted SQL files	Do not claim recursive or correlated query coverage unless the corresponding SQL statements are present and demonstrable.
Team contribution and viva	To be explained by team	Both members should be prepared to explain schema, SQL, vector search, API, and frontend contributions.

Note: The CTE/correlated/recursive line is deliberately called out for verification because the screenshots supplied here do not demonstrate those specific query forms. If the project SQL files contain them, cite the relevant script and add a terminal output capture. Do not label a scalar nested subquery as a correlated or recursive query.

11. Limitations and Future Enhancements

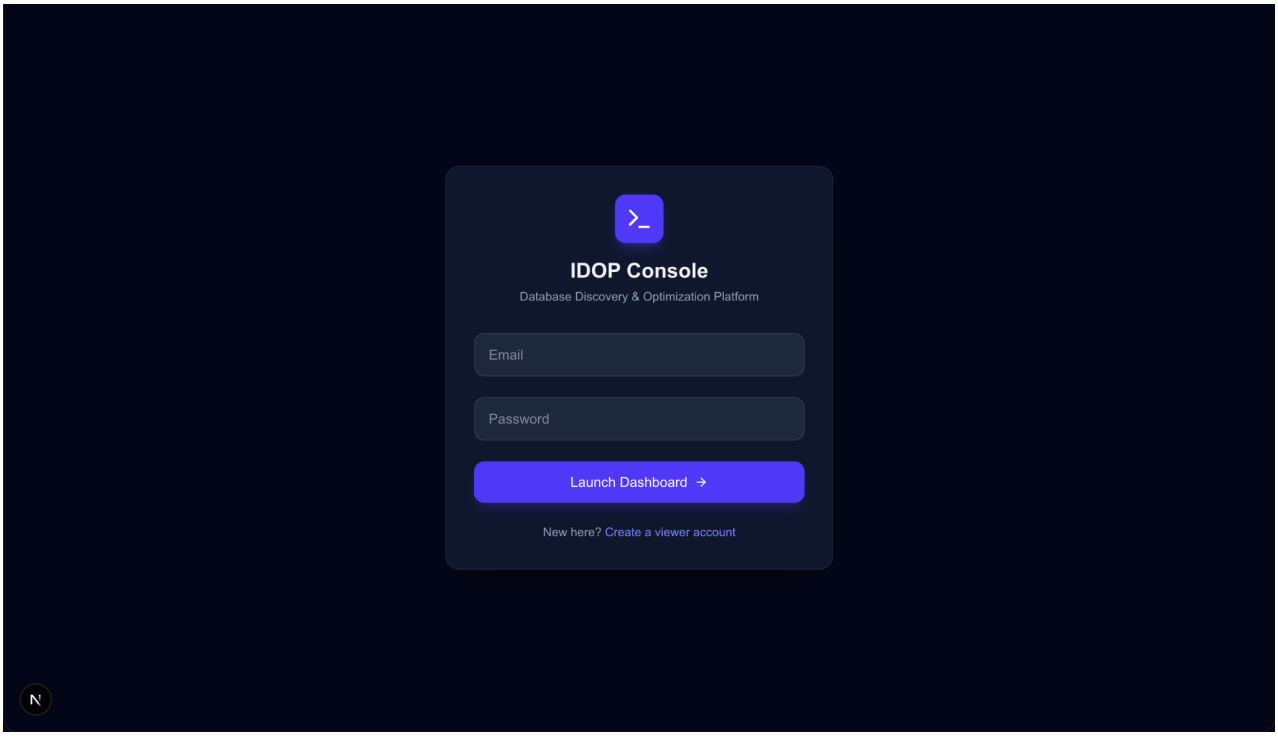
• The SQL analyzer is rule-based and does not replace a complete SQL parser.
• Optimization recommendations are advisory; the application does not automatically rewrite SQL or claim measured

speedups.
• The demonstration workload set is small and seeded for project demonstration.
• EXPLAIN output provides optimizer estimates, not a before/after runtime benchmark.
• Future work includes broader workload ingestion, a full SQL parser, measured performance experiments, and automated end-to-end integration tests.

12. Application Screenshots

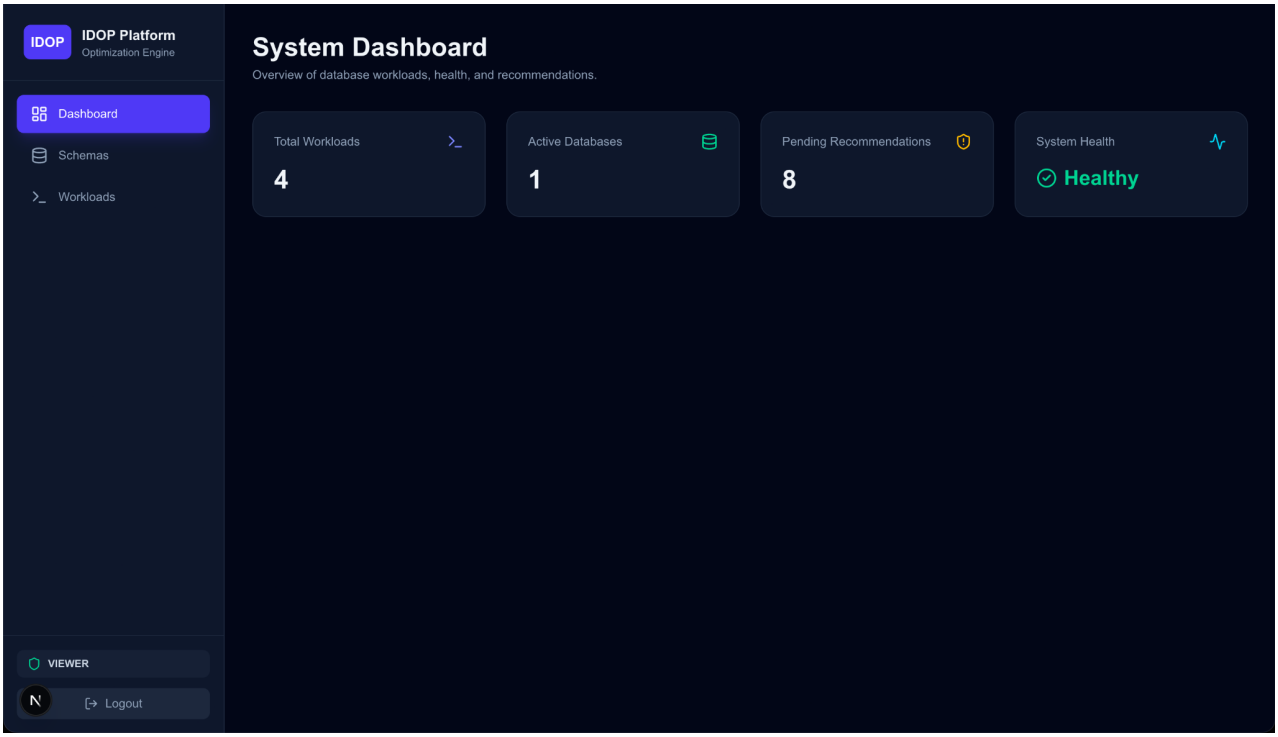
The following figures are screenshots supplied by the project team on 2 November 2026. The poorly formatted SHOW TRIGGERS capture is intentionally excluded; the audit-log result is included as evidence of trigger activity.

Figure 1 – Login interface



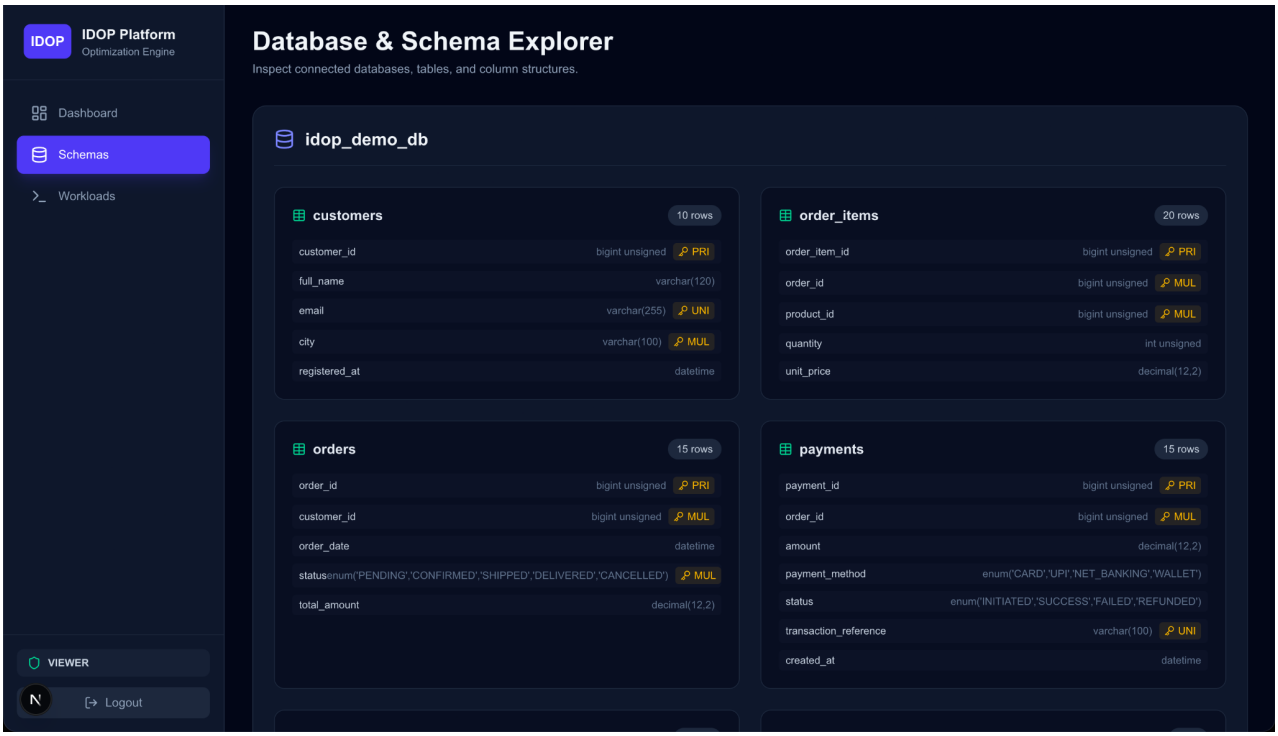
IDOP Console login page with account creation link.

Figure 2 – System dashboard



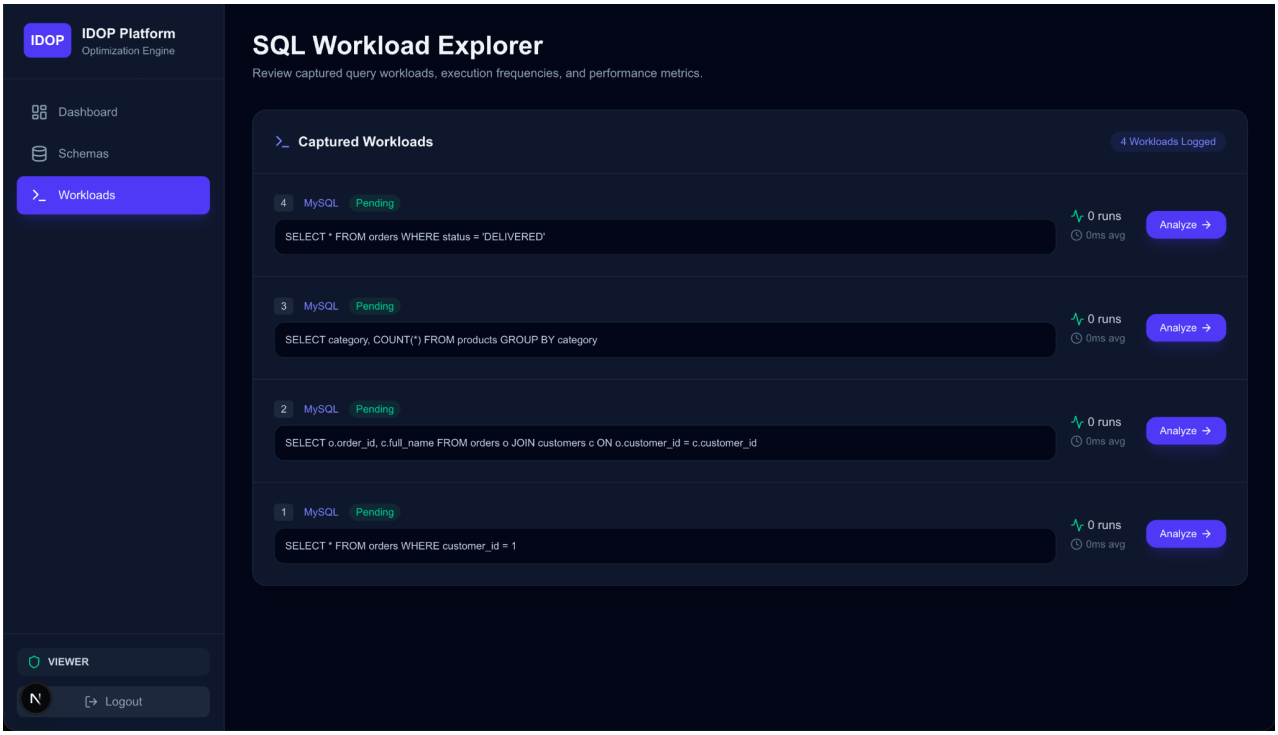
Dashboard summary showing 4 workloads, 1 active database, 8 pending recommendations, and the displayed system-health indicator.

Figure 3 – Database and schema explorer



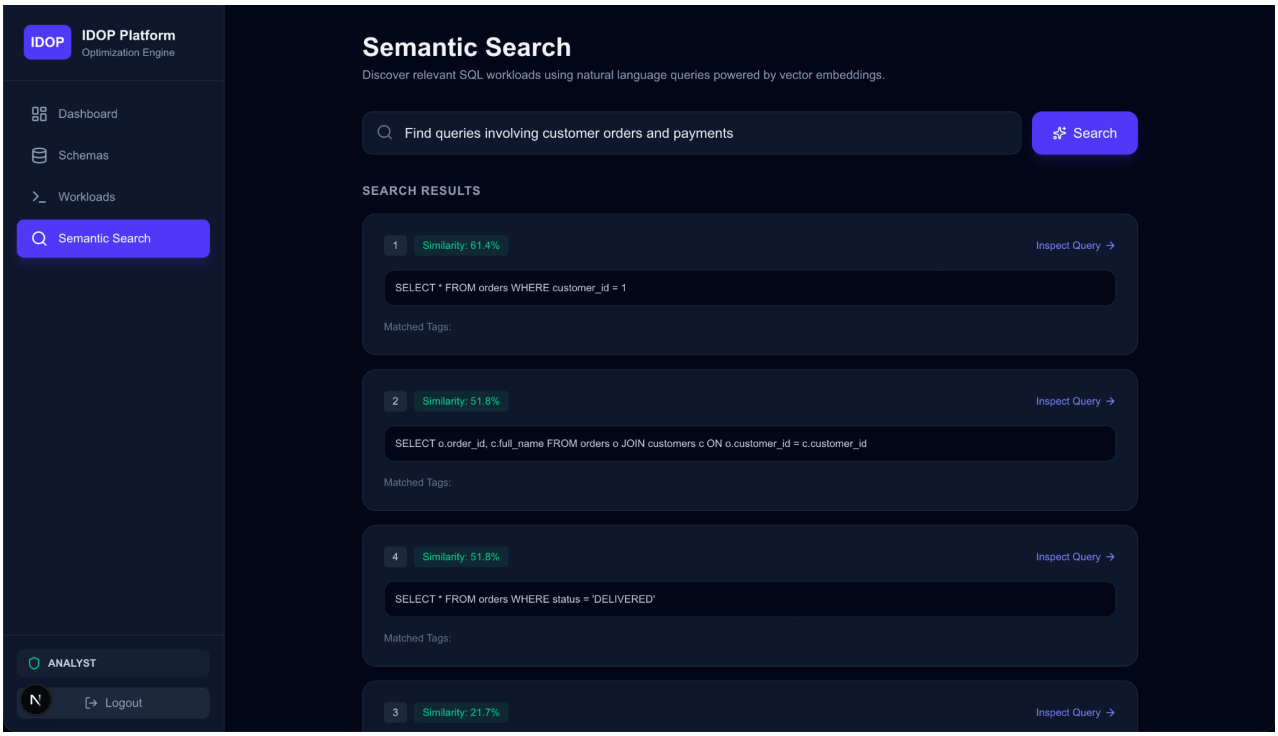
Schema explorer showing idop_demo_db tables, columns, row counts, and key indicators.

Figure 4 – SQL workload explorer



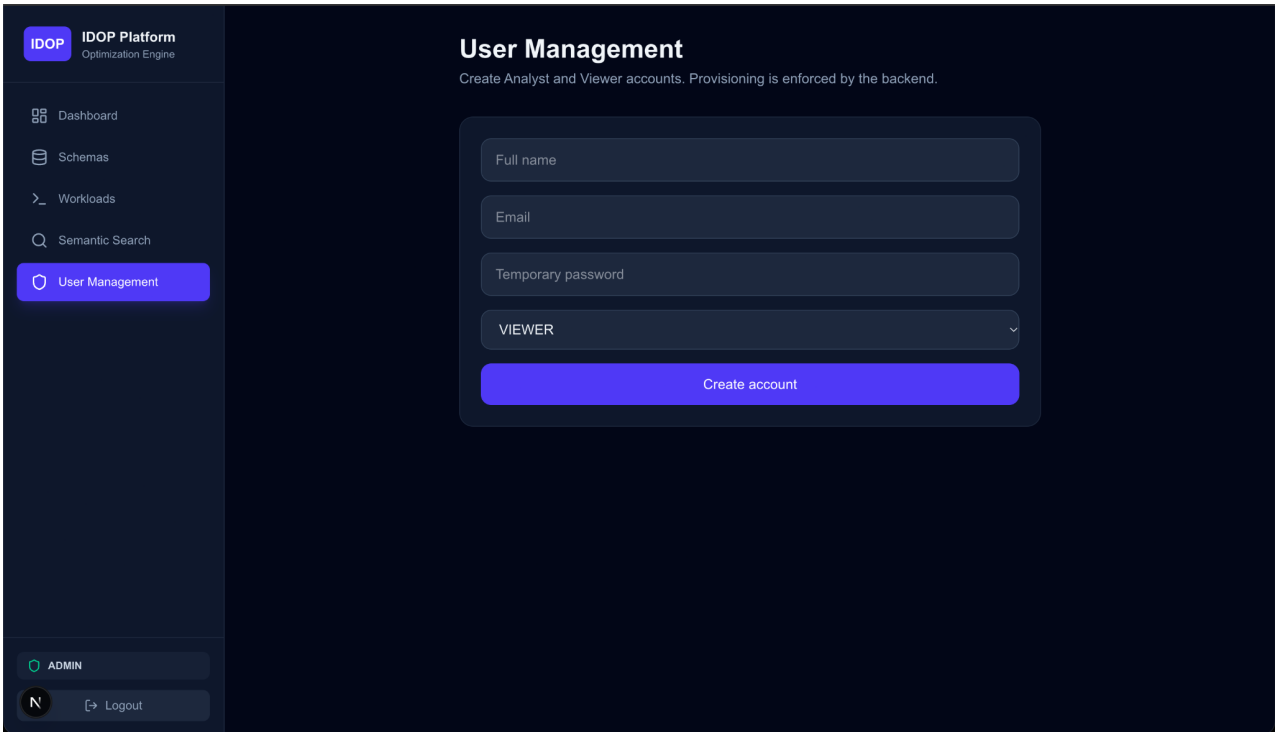
Four captured SQL workloads with SQL text and Analyze actions.

Figure 5 – Semantic search



Natural-language query with ranked workload matches and similarity percentages.

Figure 6 – User management



Admin user-management interface with role selection.

Figure 7 – MySQL database objects

```
mysql> USE idop_demo_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+--------------------------+
| Tables_in_idop_demo_db   |
+--------------------------+
| customers                |
| order_items              |
| orders                   |
| payments                 |
| products                 |
| v_customer_order_summary |
+--------------------------+
6 rows in set (0.001 sec)

mysql>
mysql> USE idop_platform_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+-----------------------------+
| Tables_in_idop_platform_db  |
+-----------------------------+
| audit_logs                  |
| data_sources                |
| optimization_recommendations |
| query_workloads             |
| schema_objects              |
| users                       |
+-----------------------------+
6 rows in set (0.001 sec)
```

SHOW TABLES output for idop_demo_db and idop_platform_db.

Figure 8 – Join and aggregation

```
mysql> USE idop_demo_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql>
mysql> SELECT
    ->     c.full_name,
    ->     COUNT(o.order_id) AS total_orders,
    ->     SUM(o.total_amount) AS total_order_value
    -> FROM customers c
    -> LEFT JOIN orders o
    ->     ON c.customer_id = o.customer_id
    -> GROUP BY c.customer_id, c.full_name;
+--------------+--------------+-------------------+
| full_name    | total_orders | total_order_value |
+--------------+--------------+-------------------+
| Aarav Sharma |            3 |          11243.00 |
| Ananya Rao   |            2 |           6197.00 |
| Rohan Mehta  |            2 |           5098.00 |
| Isha Nair    |            1 |           1299.00 |
| Kabir Verma  |            1 |           3499.00 |
| Diya Iyer    |            2 |           5697.00 |
| Arjun Reddy  |            1 |           2199.00 |
| Meera Joshi  |            1 |           1599.00 |
| Vivaan Gupta |            1 |           2398.00 |
| Saanvi Menon |            1 |            499.00 |
+--------------+--------------+-------------------+
10 rows in set (0.001 sec)
```

LEFT JOIN and aggregation query returning customer order counts and values.

Figure 9 – Stored function and procedure

```
mysql> USE idop_demo_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql>
mysql> SELECT
    ->     c.full_name,
    ->     COUNT(o.order_id) AS total_orders,
    ->     SUM(o.total_amount) AS total_order_value
    -> FROM customers c
    -> LEFT JOIN orders o
    ->     ON c.customer_id = o.customer_id
    -> GROUP BY c.customer_id, c.full_name;
+--------------+--------------+-------------------+
| full_name    | total_orders | total_order_value |
+--------------+--------------+-------------------+
| Aarav Sharma |            3 |          11243.00 |
| Ananya Rao   |            2 |           6197.00 |
| Rohan Mehta  |            2 |           5098.00 |
| Isha Nair    |            1 |           1299.00 |
| Kabir Verma  |            1 |           3499.00 |
| Diya Iyer    |            2 |           5697.00 |
| Arjun Reddy  |            1 |           2199.00 |
| Meera Joshi  |            1 |           1599.00 |
| Vivaan Gupta |            1 |           2398.00 |
| Saanvi Menon |            1 |            499.00 |
+--------------+--------------+-------------------+
10 rows in set (0.001 sec)

mysql>
mysql>
mysql>
mysql>
mysql>
mysql> SELECT idop_demo_db.fn_order_item_total(1);
+--------------------------------------+
| idop_demo_db.fn_order_item_total(1) |
+--------------------------------------+
|                              4798.00 |
+--------------------------------------+
1 row in set (0.009 sec)

mysql>
mysql> CALL idop_demo_db.sp_reconcile_order_total(1);
Query OK, 0 rows affected (0.001 sec)
```

Function result and successful stored-procedure execution.

Figure 10 – Customer summary view

```
mysql> SELECT *
    -> FROM idop_demo_db.v_customer_order_summary
    -> LIMIT 10;
+-------------+--------------+---------------------------+-------------+---------------------------+---------------------+
| customer_id | full_name    | email                     | order_count | non_cancelled_order_value | last_order_date     |
+-------------+--------------+---------------------------+-------------+---------------------------+---------------------+
|           1 | Aarav Sharma | aarav.sharma@example.com  |           3 |                  11243.00 | 2025-09-08 11:20:00 |
|           2 | Ananya Rao   | ananya.rao@example.com    |           2 |                   6197.00 | 2025-08-28 09:30:00 |
|           3 | Rohan Mehta  | rohan.mehta@example.com   |           2 |                   5098.00 | 2025-09-02 12:45:00 |
|           4 | Isha Nair    | isha.nair@example.com     |           1 |                      0.00 | 2025-08-10 17:10:00 |
|           5 | Kabir Verma  | kabir.verma@example.com   |           1 |                   3499.00 | 2025-08-12 11:00:00 |
|           6 | Diya Iyer    | diya.iyer@example.com     |           2 |                   3798.00 | 2025-09-05 15:35:00 |
|           7 | Arjun Reddy  | arjun.reddy@example.com   |           1 |                   2199.00 | 2025-08-18 16:25:00 |
|           8 | Meera Joshi  | meera.joshi@example.com   |           1 |                   1599.00 | 2025-08-20 10:50:00 |
|           9 | Vivaan Gupta | vivaan.gupta@example.com  |           1 |                   2398.00 | 2025-08-22 14:15:00 |
|          10 | Saanvi Menon | saanvi.menon@example.com  |           1 |                    499.00 | 2025-08-25 18:00:00 |
+-------------+--------------+---------------------------+-------------+---------------------------+---------------------+
10 rows in set (0.001 sec)
```

SELECT from v_customer_order_summary showing customer-level aggregates.

Figure 11 – Trigger audit result

```
mysql> SELECT recommendation_id, status
    -> FROM optimization_recommendations
    -> LIMIT 5;
+-------------------+--------+
| recommendation_id | status |
+-------------------+--------+
|                 3 | NEW    |
|                 4 | NEW    |
|                 6 | NEW    |
|                 7 | NEW    |
|                 9 | NEW    |
+-------------------+--------+
5 rows in set (0.001 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.000 sec)

mysql>
mysql> UPDATE optimization_recommendations
    -> SET status = 'REVIEWED'
    -> WHERE recommendation_id = 1;
Query OK, 0 rows affected (0.001 sec)
Rows matched: 1  Changed: 0  Warnings: 0

mysql>
mysql> SELECT *
    -> FROM audit_logs
    -> ORDER BY audit_id DESC
    -> LIMIT 5;
+----------+---------+-----------------------------+---------------------------+-----------+-----------------------------------------------------------------------------------------------------------------------------------------------------------------------+---------------------+
| audit_id | user_id | action                      | entity_type               | entity_id | details                                                                                                                                                               | created_at          |
+----------+---------+-----------------------------+---------------------------+-----------+-----------------------------------------------------------------------------------------------------------------------------------------------------------------------+---------------------+
|        1 |    NULL | RECOMMENDATION_STATUS_CHANGED | optimization_recommendations |         1 | {"rule_code": "IDX_FILTER_COLUMN", "new_status": "REVIEWED", "old_status": "NEW"} | 2026-11-01 21:50:10 |
+----------+---------+-----------------------------+---------------------------+-----------+-----------------------------------------------------------------------------------------------------------------------------------------------------------------------+---------------------+
1 row in set (0.000 sec)

mysql>
mysql> ROLLBACK;
Query OK, 0 rows affected (0.000 sec)
```

Audit log entry for RECOMMENDATION_STATUS_CHANGED with old and new status details.

Figure 12 – MySQL FULLTEXT search

```
mysql> SELECT
    ->     workload_id,
    ->     query_label,
    ->     MATCH(query_label, sql_text)
    ->     AGAINST ('customer orders' IN NATURAL LANGUAGE MODE) AS relevance
    -> FROM query_workloads
    -> WHERE MATCH(query_label, sql_text)
    ->       AGAINST ('customer orders' IN NATURAL LANGUAGE MODE)
    -> ORDER BY relevance DESC;
+-------------+-------------------------------+----------------------+
| workload_id | query_label                   | relevance            |
+-------------+-------------------------------+----------------------+
|           1 | Orders by customer            |    0.393695592880249 |
|           2 | Orders joined with customers | 0.031219376251101494 |
|           4 | Filter delivered orders       | 0.031219376251101494 |
+-------------+-------------------------------+----------------------+
3 rows in set (0.005 sec)
```

FULLTEXT query returns three matching workload records with relevance values.

Figure 13 – MySQL EXPLAIN

```
mysql> USE idop_demo_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql>
mysql> EXPLAIN
    -> SELECT *
    -> FROM orders
    -> WHERE customer_id = 1;
+----+-------------+--------+------------+------+---------------------------+---------------------------+---------+-------+------+----------+-------+
| id | select_type | table  | partitions | type | possible_keys             | key                       | key_len | ref   | rows | filtered | Extra |
+----+-------------+--------+------------+------+---------------------------+---------------------------+---------+-------+------+----------+-------+
|  1 | SIMPLE      | orders | NULL       | ref  | idx_orders_customer_date | idx_orders_customer_date | 8       | const |    3 |   100.00 | NULL  |
+----+-------------+--------+------------+------+---------------------------+---------------------------+---------+-------+------+----------+-------+
1 row in set, 1 warning (0.002 sec)
```

Execution plan showing use of idx_orders_customer_date.

Figure 14 – Transaction rollback

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.000 sec)

mysql>
mysql> UPDATE orders
    -> SET status = 'CANCELLED'
    -> WHERE order_id = 8;
Query OK, 1 row affected (0.001 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> SELECT order_id, status
    -> FROM orders
    -> WHERE order_id = 8;
+----------+-----------+
| order_id | status    |
+----------+-----------+
|        8 | CANCELLED |
+----------+-----------+
1 row in set (0.000 sec)

mysql>
mysql> ROLLBACK;
Query OK, 0 rows affected (0.001 sec)

mysql>
mysql> SELECT order_id, status
    -> FROM orders
    -> WHERE order_id = 8;
+----------+---------+
| order_id | status  |
+----------+---------+
|        8 | PENDING |
+----------+---------+
1 row in set (0.001 sec)
```

Order 8 changes to CANCELLED within the transaction and returns to PENDING after ROLLBACK.

Figure 15 – Qdrant collection

```
~/Documents/DBMS-mini-project  main                                  5m 26s  12:36:29 PM
curl -s http://localhost:6333/collections/sql_workloads
{"result":{"status":"green","optimizer_status":"ok","indexed_vectors_count":0,"points_count":4,"segments_count":5,"config":{"params":{"vectors":{"size":384,"distance":"Cosine"},"shard_number":1,"replication_factor":1,"write_consistency_factor":1,"on_disk_payload":true},"hnsw_config":{"m":16,"ef_construct":100,"full_scan_threshold":10000,"max_indexing_threads":0,"on_disk":false},"optimizer_config":{"deleted_threshold":0.2,"vacuum_min_vector_number":1000,"default_segment_number":0,"max_segment_size":null,"memmap_threshold":null,"indexing_threshold":10000,"flush_interval_sec":5,"max_optimization_threads":null,"prevent_unoptimized":null},"wal_config":{"wal_capacity_mb":32,"wal_segments_ahead":0,"wal_retain_closed":1},"quantization_config":null},"payload_schema":{},"update_queue":{"length":0},"created_at":"2026-10-25T15:32:13.011Z"},"status":"ok","time":0.001014458}

~/Documents/DBMS-mini-project  main                                  12:36:31 PM
curl -s -X POST \
  http://localhost:6333/collections/sql_workloads/points/count \
  -H "Content-Type: application/json" \
  -d '{"exact":true}'
{"result":{"count":4},"status":"ok","time":0.000725375}
```

Qdrant collection response and exact point count: four vectors, 384 dimensions, cosine distance, healthy status.

13. Repository and Run Instructions

GitHub repository: https://github.com/PiyushShiv26/DBMS-mini-project

Configure local environment variables using the example environment file. Do not commit database passwords, API secrets, or provisioning keys.

```
# From repository root – backend
source .venv/bin/activate
uvicorn backend.main:app --reload --port 8000

# Frontend – in another terminal
cd frontend
npm install
npm run dev
```

Start MySQL and Qdrant before launching the application. Qdrant's local API is available on port 6333 when using the documented Docker configuration.

End of Report